

Do Early Internal Trajectories Reveal Terminal Reward?

TENNY LIU, Center for Data Science, New York University, USA

Executive Summary

Motivation. How do pretrained language models organize generation in ways that are informative about how their behavior in post-training? One of the foundational disputes in RL for reasoning models is: whether RL mainly *sharpens* or reweights capabilities already present in the pretrained policy, or *discovers* behaviors that resembles generalized knowledge outside its prior repertoire.

Two behavioral results make the hypothesis of a pretraining-conditioned trajectory divergence plausible. Gandhi et al. [9] find that initial verification, backtracking, subgoal-setting, and backward-chaining behaviors help explain why Qwen improves more than Llama under the same RL procedure. Priming Llama with model-specific behavioral patterns, even in incorrect solutions, or increasing them through filtered continued pretraining improved its subsequent RL trajectory. Shao et al. [20] finds that weak and random rewards can improve some Qwen models, while usually failing on the tested Llama and OLMo models.

Together, these studies show that the starting policy can shape what RL amplifies and predetermines how much it improves in their tested settings, linking observable starting behaviors and training dynamics to expected future reward. This work aims to identify whether such reward signals are hidden in model representations. Since the different starting/pre-trained models are difficult to measure across model families, this work narrows to a testable question: whether the residual state and token-wise confidence of an early generated sequence contains information that predicts its terminal outcome across pre- and post-trained variants of the same model.

Experiment. Zhang et al. [23] highlights several open challenges in RL for reasoning language models: reward design, policy optimization, and rollout sampling strategy. It poses the central challenge of objective specification: designing the *reward function* R , or the verifier/reward model that implements it.

In this experiment, the requirements are stated in a natural language prompt, while IFEvalG implements R_T as the fraction of executable constraint checks that pass. It’s a tractable instruction-following proxy that does not measure factual accuracy, usefulness, or the user’s complete intent. However, the complete OLMo post-training datasets contain benchmarks for other verifiable reasoning tasks like math and code.

For prompt x_i , let c_{ij} denote its j th executable IFEvalG constraint and let m_i be the number of constraints. The terminal reward for a completed response y is

$$R_T(x_i, y) = \frac{1}{m_i} \sum_{j=1}^{m_i} \mathbb{I}[c_{ij}(x_i, y) = 1]. \quad (1)$$

For model stage $s \in \{\text{Base, SFT, DPO, RLVR}\}$, the continuation value after generated token t is

$$V^{\pi_s}(x_i, y_{\leq t}) = \mathbb{E}_{y_{>t} \sim \pi_s(\cdot | x_i, y_{\leq t})} [R_T(x_i, y_{\leq t} \circ y_{>t})]. \quad (2)$$

This is the expected fraction of constraints that will be satisfied after the model completes the response from the current generated text. At layer ℓ , the model’s residual state is

$$h_{i,t}^{(s,\ell)} = f^{(s,\ell)}(x_i, y_{\leq t}). \quad (3)$$

Because this state is computed from the tokenized prompt and all generated tokens through position t , it is a candidate representation of the current generation state. Its contents, however, cannot be assumed in advance. The main question is whether $h_{i,t}^{(s,\ell)}$ predicts continuation value beyond information already available from prompt identity, visible text, partial-verifier status, token identity, generated length, and policy confidence.

The completed pilot observes only one continuation from each state. Its target is therefore the realized reward

$$r_{isk} = R_T(x_i, y^{(s,k)}), \quad (4)$$

rather than the expectation $V^{\pi_s}(x_i, y_{\leq t})$. A future experiment will sample K independent continuations from an identical generated sequence and estimate

$$\widehat{V}^{\pi_s}(x_i, y_{\leq t}) = \frac{1}{K} \sum_{k=1}^K R_T(x_i, y_{\leq t} \circ y_{>t}^{(k)}). \quad (5)$$

Results. I evaluated OLMo 3 Base, SFT, DPO, and RLVR on a subset of instruction-following prompts, with five stochastic rollouts per prompt and checkpoint. IFEvalG supplied constraint-level terminal rewards. Residual states from layers 8, 16, 24, and 32 and token-level confidence were recorded after 25%, 50%, 75%, and 100% of the 128 generated tokens. Model variant, layer, and observed generated length indexed the comparisons. Terminal verifier reward was the outcome; activation and confidence measurements were candidate predictors observed at different points in the trajectory.

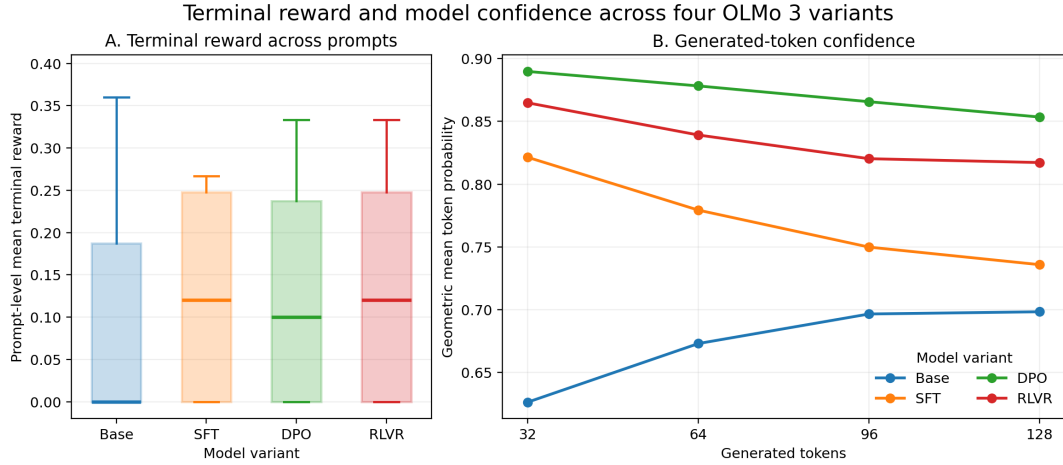


Fig. 1. Descriptive outcomes from the completed 120-rollout experiment. All four OLMo 3 model variants are shown. Aggregate reward changed little, whereas generated-token confidence followed distinct trajectories across Base, SFT, DPO, and RLVR.

Post-trained variants were more confident than Base at every recorded generation position, but it clearly does not correlate with satisfying IFEvalG.

Final-layer activation movement also differed across variants, and distance traveled in residual space has no single reward-oriented direction.

Trajectory measurements have prompt-sensitive associations with terminal reward

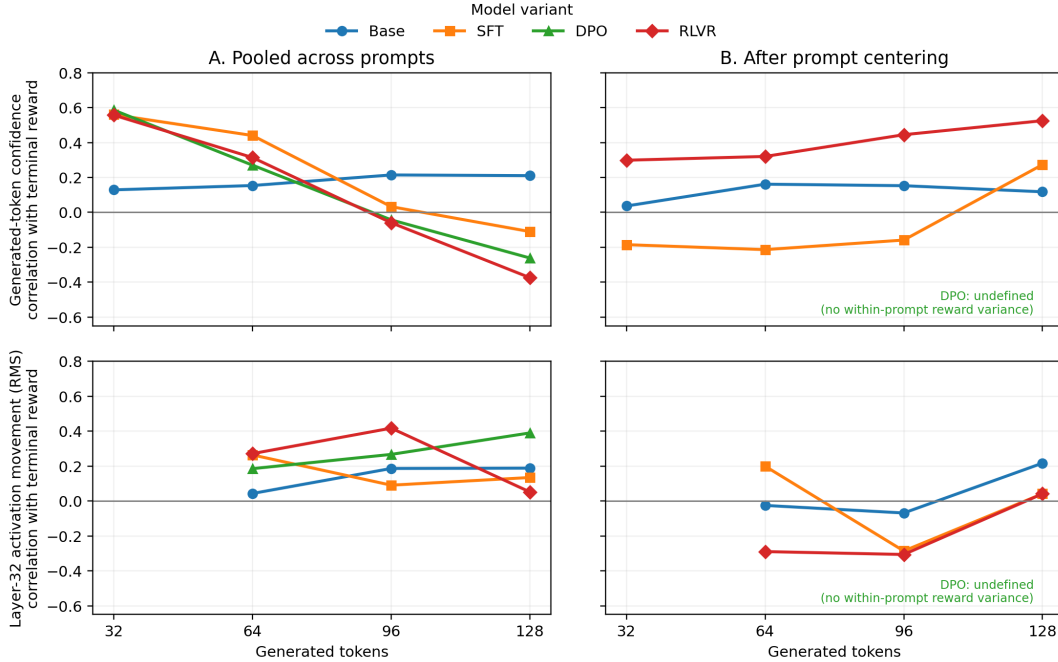


Fig. 2. Pearson correlations between trajectory measurements and terminal reward across four Olmo 3 model variants. Left panels pool all prompts. Right panels isolate variation among repeated rollouts of the same prompts.

Conclusion and Research Plan. This exploration suggests two complementary directions for intermediate evaluation in language-model reinforcement learning.

First, within a rollout of a fixed model checkpoint, intermediate evaluation should account for which outcomes remain reachable from the current generation. This is particularly clear for constraints such as using a keyword exactly a specified number of times or satisfying a global formatting requirement, which may not be definitively evaluable until termination.

From a prefix τ_t , I can sample K independent continuations under a fixed remaining generation budget B_t and estimate

$$\hat{p}_c(\tau_t, B_t) = \frac{1}{K} \sum_{k=1}^K \mathbb{I}[\text{constraint } c \text{ is satisfied in continuation } k]. \quad (6)$$

This provides an empirical estimate of the outcomes still reachable under the current policy, rather than treating the reward of one sampled completion as the only target. Similar rollout-based estimates are well-studied in games and rule-based RL; an important extension is to define reliable success criteria for coding, mathematical, and semantic tasks with

verifiers, exact-answer checks, or calibrated LLM judges. Datasets such as allenai/RLVR-GSM-MATH-IF-Mixed-Constraints provide a useful setting.

The mechanistic question remains: do hidden states or confidence predict this continuation-based estimate of future success? The present results do not demonstrate that model activations add predictive information beyond observable rewards. It does, however, indicate that activation dynamics and model confidence should not be treated as interchangeable signals.

Second, across training checkpoints within the same model variant, the relevant question is how these quantities change with respect to the starting policy. Task difficulty, confidence calibration, the point at which terminal outcomes become predictable, and the internal representations associated with those outcomes can all be compared across within Base, SFT, DPO, and RLVR checkpoints. The goal is still to identify the source of how pretraining already organize partial solutions in ways related to later success for certain tasks and prompts.

Related Work.

Internal feedback and confidence-based rewards. Several recent methods use quantities derived from the model itself as reinforcement-learning signals. Intuitior uses self-certainty as internal feedback, while RENT and related entropy-minimization approaches reward confident or low-entropy reasoning [1, 19, 25]. These results show that model-derived signals can influence learning, but confidence is not equivalent to externally defined task success. A model can become increasingly certain while remaining wrong, and continued optimization of an internal proxy can diverge from the intended objective [24]. The present work therefore treats confidence as a measured predictor of future success rather than assuming that it constitutes an appropriate reward.

Inverse reinforcement learning and reward inference. Inverse reinforcement learning addresses the complementary problem of inferring a reward function under which observed behavior would appear likely or optimal [17, 27]. The current setting differs because the external verifier reward is already specified. The immediate problem is therefore closer to value estimation: whether information about expected future reward is recoverable from an intermediate policy state. An inverse-RL extension could reverse the analysis by asking what objective is implied by the model’s behavior or internal features and whether that inferred objective agrees with different verifiers or human-specified criteria.

1 Introduction

Reinforcement learning for language models commonly assigns reward after a complete response has been generated. This makes credit assignment difficult: the score says whether the outcome met an objective, but not when the model’s trajectory became promising or went wrong. Process labels can supply denser supervision, but they are expensive and may impose an evaluator’s decomposition of the task. A complementary possibility is that pretraining has already organized linguistic goals, partial solutions, uncertainty, and plans in a representation from which expected outcome is recoverable.

This possibility is more specific than treating generation as an ordinary finite-state control problem. The value of a partial response depends on the prompt: the same words can satisfy one requested constraint and violate another. In the present experiment, however, the prompt is not supplied to the predictor as a separate goal variable. It is already part of the context used to compute h_t , so the measured question is whether h_t predicts R_T . A future design could explicitly separate a goal representation g and fit $V(h_t, g)$ across systematically varied instructions. If such a function were smooth enough to estimate, $\nabla_{h_t} V$ would give a local sensitivity direction for later intervention. The present pilot asks only the prior observational question of whether outcome information is measurably present.

This work studies that distinction in a small, controlled instruction-following setting. IFEval-style prompts are useful because they attach deterministic, programmatically checkable constraints to natural-language responses [26]. I compare four publicly released checkpoints from the same OLMo model family, reducing architectural differences relative to a comparison of unrelated models [3, 10].

The study asks three questions:

- (1) How do hidden-state movement and policy confidence evolve during a rollout, and how are they associated with terminal verifier reward?
- (2) Can intermediate internal features predict future reward beyond the prompt and constraint progress already observable in the partial text?
- (3) How do these relationships differ across Base, SFT, DPO, and RLVR checkpoints?

These questions concern *internal-behavioral correspondence*. Even perfect held-out prediction would establish only that reward is decodable under a particular data split. It would not show that the model uses the decoded direction, that the signal estimates future reward rather than restating visible prefix content, or that it transfers across tasks and model stages. Evidence for those stronger claims would require incremental prediction over observable controls, robustness across prompt families, matched-prefix comparisons, and ultimately intervention. Similarly, the comparison across checkpoints is descriptive. The present data do not include controlled unseen task variants and therefore do not test whether SFT memorizes or whether RL generalizes.

2 Related Work

2.1 From Outcome Reward to Prefix-Level Value

Outcome-supervised RL assigns credit from a final answer, whereas process reward models score individual reasoning steps. Lightman et al. [16] train process models from human step labels and use them to select solutions. ReST-MCTS instead expands reasoning trees from intermediate steps, uses final answer correctness to estimate the probability that a step leads to success, and reuses those estimates both as process-value targets and to select traces for self-training [22]. These systems construct prefix-level supervision outside the policy. My question is complementary: before training

another process evaluator, how much future-outcome information is already available in the policy’s own contextual representation?

RE-Control is especially close to the longer-term objective of this work. It models an autoregressive LLM as a stochastic dynamical system, fits a Bellman value function directly on hidden states, and differentiates that value to optimize test-time control vectors [14]. This demonstrates how a differentiable hidden-state value can become actionable. It does not by itself establish that an unmodified pretrained model naturally represents a general value signal, or that a fitted gradient corresponds to the model’s own causal computation. The present pilot keeps prediction, gradient attribution, and intervention as separate evidential steps.

2.2 Pretrained Behavioral Priors and RL Amplification

The closest motivation for the present hypothesis comes from work asking why the same RL procedure improves some starting models more than others. Gandhi et al. [9] compare Qwen-2.5-3B and Llama-3.2-3B on Countdown and characterize four observable reasoning behaviors: verification, backtracking, subgoal setting, and backward chaining. Qwen exhibited more of these behaviors before RL and improved more. The authors then intervened on the starting policy: behavioral priming, including priming with incorrect answers that retained the reasoning patterns, and filtered continued pretraining both made Llama more responsive to subsequent RL. This supports the claim that the initial behavioral repertoire can enable self-improvement. It does not show that these behaviors are represented as value, nor that a hidden state predicts the outcome of a particular continuation.

Shao et al. [20] push the prior-amplification account further. On Qwen2.5-Math, GRPO training with format, incorrect, or random rewards produced substantial MATH-500 gains, while comparable spurious signals generally failed to help the tested Llama and OLMo2 models. Their analysis identifies a clipping bias that favors high-probability tokens and tracks the amplification of pre-existing code-reasoning behavior in Qwen. The result does not imply that reward content is generally irrelevant: the effect is model- and distribution-dependent, ground-truth rewards can improve through different mechanisms, and the reported spurious-reward gains largely weaken on the more out-of-distribution AIME 2025 evaluation. It does show that an apparent RL improvement can arise from the interaction among optimizer, model prior, and evaluation distribution rather than from reward semantics alone.

These studies motivate a more specific mechanistic question. Behavioral analysis can establish *what* patterns a base model emits and training experiments can establish *which* patterns RL amplifies. The present work asks whether, before that amplification, the model’s contextual residual state contains incremental information about the expected terminal outcome of its current partial trajectory. A positive result would connect behavioral priors to prefix-level outcome prediction; it would still not prove that the state is a learned reward, a causal value computation, or a representation of human intent.

2.3 Internal Feedback and Entropy-Based Reward

Recent work asks whether a model’s own uncertainty can replace external supervision in RL for reasoning [23]. Intuitior substitutes self-certainty for the external reward in GRPO, while RENT reinforces reasoning traces that yield low-entropy, high-confidence answers [19, 25]. These methods optimize an internal statistic rather than first requiring labeled solutions. However, Zhang et al. [24] report that gains from internal feedback can diminish or reverse during training and can be limited for instruction-tuned models. My experiment tests a narrower prerequisite: whether confidence during generation corresponds to an independently specified terminal verifier.

Cui et al. [6] are concrete about how entropy changes under RL. They derive token-level entropy change from the covariance between action probability and logit updates, relate those updates to advantage, and introduce Clip-Cov and KL-Cov to restrict updates on high-covariance tokens. Their covariance is a training-dynamics quantity, not covariance of residual-stream activations. This distinction matters here: entropy, confidence, value, and activation geometry may interact, but none can be substituted for another without an empirical calibration against future outcomes.

2.4 Reward Hacking and Trajectory Branching

A reward can be perfectly measured and still be misspecified: optimizing the measured proxy may produce behavior that violates the intended task. This is why terminal score alone is insufficient evidence of desirable behavior. In a recent open-ended case study, Kroiz et al. [15] analyze frontier models acting in chess and tic-tac-toe environments where hacking is possible. They treat agent actions as branching points and resample later actions to estimate how an earlier choice changes the probability of eventual hacking. For o3, attempting to inspect the environment source code increased subsequent hacking under this intervention; the same action-level result did not hold for GPT-5. Their study concerns behavioral actions in closed models, whereas the present project observes token-position residual states in an open model. The shared methodological lesson is that repeated futures from a matched earlier state provide stronger evidence than correlating one observed trajectory with its endpoint. A future reward-hacking extension of this project would also need two labels—proxy reward and intended task success—because predicting the proxy alone cannot diagnose exploitation of it.

2.5 Inverse RL, Language, and Reward Design

Inverse reinforcement learning infers a reward function that makes observed expert behavior appear optimal [17, 27]. It is related to this project at the level of reward identification, but the observed object differs. Classical IRL starts from demonstrations and asks which external reward explains them; this project starts from an LLM’s internal trajectory and asks whether it contains information about a separately measured terminal objective.

Language-conditioned reward work illustrates why an LLM representation may be more than a generic low-dimensional Markov state. Text2Reward translates a natural-language goal and an environment description into executable dense reward code, then refines that program [21]. Here, rather than asking an LLM to write a reward function, I test whether a residual state computed from the instruction and generated history is informative about an external terminal score. An explicitly goal-conditioned value $V(h_t, g)$ is a future extension requiring systematically varied instructions and a separate encoding of g ; it is not what the present probe implements.

2.6 Representation, Probing, and Post-Training

Prompt representations can predict global attributes of later responses, including structure and content [7]. That motivates looking before the final token, but prediction of a future attribute is not yet evidence of a value function. Control tasks distinguish encoded information from properties learned by a flexible probe [13]; conditional probing asks what is available beyond a baseline [12]; and amnesic probing shows that decodability need not imply behavioral use [8].

Post-training can also alter how a behavior is internally implemented. Nguyen et al. [18] compare supervised, reasoning-augmented, and preference-based refusal training across three model architectures, then study the resulting refusal mechanisms and steering robustness. Their result motivates matched-prefix comparisons across OLMo stages: if RL amplifies or reorganizes pretrained outcome-relevant features, checkpoints should be compared on identical

linguistic states rather than only on responses each checkpoint generated for itself. Chu et al. [5] provide a separate behavioral motivation for held-out evaluation: on the constructed GeneralPoints card task and V-IRL navigation task, they compare SFT and outcome-based RL on unseen textual and visual rule variants and find stronger generalization after RL in that setting. They do not study residual-stream value representations, early prediction of terminal reward, or OLMo checkpoints. Their paper therefore motivates the use of controlled unseen variants here, but it is not direct mechanistic evidence for the present hypothesis. This training-source pilot cannot yet test their generalization claim.

3 Methods

3.1 Experimental Design and Checkpoints

The experiment used four OLMo 3 7B checkpoints:

- **Base:** allenai/OLMo-3-1025-7B;
- **SFT:** allenai/OLMo-3-7B-Think-SFT;
- **DPO:** allenai/OLMo-3-7B-Think-DPO; and
- **RLVR:** allenai/OLMo-3-7B-Think.

For every checkpoint, the model generated five stochastic rollouts for each of six prompts. Generation used temperature 0.6, top- p 0.95, random seed 0, and a maximum of 128 new tokens.

3.2 Prompt Source

Prompts were streamed from revision 0fb6466d31ef3a9dd16985ef635e6429e05a6491 of allenai/Dolci-Think-RL-7B. The loader selected examples whose original_dataset field was hamishivi/IF_multi_constraints_upto5_filtered. This source was part of the data used for OLMo 3 RLVR training [2]. Consequently, the experiment measures behavior on an RLVR training-source distribution, not on a held-out task distribution.

3.3 Terminal Reward

Each completed response was evaluated with the IFEvalG instruction registry from Ai2’s open-instruct implementation [4]. If a prompt specifies m constraints, terminal reward is

$$R_T = \frac{1}{m} \sum_{j=1}^m \mathbb{I}[\text{constraint}_j \text{ is satisfied}]. \quad (7)$$

The analysis retained both R_T and the binary result for every individual constraint. Reward therefore measures the fraction of requested formatting or lexical constraints satisfied; it is not a general measure of factual quality.

3.4 Trajectory Measurements

After generation, the prompt and completed response were passed through the same checkpoint again with hidden-state output enabled. Measurements were taken at the generated token nearest each trajectory fraction

$$t/T \in \{0.25, 0.50, 0.75, 1.00\}. \quad (8)$$

Residual-stream vectors were retained from transformer layers $\ell \in \{8, 16, 24, 32\}$. Each saved activation is a single-token vector, not a mean-pooled response representation. More precisely, if the tokenized prompt has length P , the response has T generated tokens, and $j_f = \text{round}(fT)$, the extracted vector is $h_{P+j_f-1}^{(\ell)} \in \mathbb{R}^{4096}$: the layer- ℓ residual

stream after the model has consumed generated token j_f . Thus a single rollout contributes $4 \text{ positions} \times 4 \text{ layers} = 16$ residual vectors. For descriptive plots, final-layer movement relative to the 25% position was measured as

$$D_t = \frac{\|h_t^{(32)} - h_{0.25T}^{(32)}\|_2}{\sqrt{d}}. \quad (9)$$

Thus $D_{0.25T} = 0$ by construction. For predictive analysis, the implementation also formed adjacent differences $\Delta h_t = h_t - h_{t-1}$.

For every generated token, the implementation computed chosen-token log probability, predictive entropy, and top-1 next-token probability. Prefix means and last-token values were saved at each trajectory fraction. Descriptive confidence was the geometric mean sampled-token probability

$$C_t = \exp \left(\frac{1}{t} \sum_{i=1}^t \log p_\theta(y_i \mid x, y_{<i}) \right). \quad (10)$$

This is a policy sharpness measure, not a calibrated probability that the response will satisfy the verifier.

3.5 Descriptive Analyses

For each checkpoint and trajectory fraction, Pearson and Spearman correlations were computed for C_t versus R_T , D_t versus R_T , and C_t versus D_t . Constraint pass rates were calculated separately by verifier instruction ID and checkpoint. These statistics pool the 30 rollouts from each checkpoint; because five rollouts share each prompt, they are treated as descriptive rather than as independent inferential observations.

3.6 Held-Out Reward Probes

Ridge-regression probes were fit separately for each checkpoint, layer, and trajectory fraction. The implemented feature sets were:

- (1) current activation h_t ;
- (2) concatenated activation history $[h_{0.25T}; \dots; h_t]$;
- (3) adjacent activation change Δh_t ;
- (4) six policy-confidence statistics; and
- (5) current activation combined with confidence statistics.

The Base checkpoint served as the cross-stage experimental reference, while prompt-block label shuffling served as the probe’s statistical null. One fixed 75/25 split was grouped by prompt, placing four prompt groups in the training set and two in the test set. All five rollouts from a prompt remained in the same split. The statistical null permuted complete prompt-level blocks of training rewards before refitting each probe. Recorded metrics included held-out R^2 , mean squared error, rank correlation, and the prompt-block permutation distribution. Each checkpoint-specific probe nevertheless had only 20 training responses, compared with 4,096 current-state features and as many as 16,384 activation-history features. Regularization makes estimation possible, but does not make training fit or a single favorable split scientifically diagnostic. Probe evidence was therefore required to show non-degenerate held-out discrimination and improvement over observable controls; the completed data did not meet those requirements.

Table 1. Terminal verifier reward across 30 rollouts per checkpoint. Standard deviations describe rollout-level variation and do not adjust for clustering by prompt.

Checkpoint	Mean	SD	Minimum	Maximum
Base	0.1017	0.1534	0.0000	0.4000
SFT	0.1261	0.1438	0.0000	0.4000
DPO	0.1306	0.1386	0.0000	0.3333
RLVR	0.1372	0.1466	0.0000	0.4000

4 Results

4.1 Terminal Reward Varied More by Prompt Than by Checkpoint

Table 1 reports aggregate verifier reward. Mean reward rose from 0.1017 for Base to 0.1372 for RLVR, an absolute difference of 0.0355. The increase was not broad across prompts. Three prompts received reward zero for every checkpoint, and one prompt received exactly 0.25 for every checkpoint. Prompt 639-88 improved from 0.0000 for Base to 0.3333 for DPO and RLVR, whereas prompt 721-31 decreased from 0.3600 for Base to 0.2400 for RLVR. Figure 1 displays this heterogeneity.

At the prompt level, the RLVR-minus-Base differences were positive for one prompt, negative for one, and exactly zero for four. The mean paired difference was 0.0356, but its prompt-clustered bootstrap interval was $[-0.060, 0.167]$ and the exact paired sign-flip p -value was 1.0. SFT and DPO showed the same one-positive, one-negative, four-unchanged pattern, with intervals that also included zero. These summaries do not demonstrate a checkpoint-level improvement; they show that the aggregate ordering can be produced by two influential prompts.

All 120 generations contained exactly 128 generated tokens. The maximum-token limit therefore censored every response, including responses evaluated with completion-sensitive constraints. Strictly, the recorded R_T is a verifier score at an imposed cutoff, not reward at a model-selected terminal state. This distinction is especially consequential for final-word, exact-length, and complete-structure requirements.

4.2 Reward Was Nearly Determined by Prompt Identity

Only four of the 24 checkpoint-by-prompt cells contained more than one reward value across their five stochastic rollouts. DPO had no within-prompt reward variation at all; Base and RLVR each had one varying prompt, and SFT had two. Predicting each rollout with its prompt’s mean reward accounted for 95.3% of reward variance for Base, 79.8% for SFT, 100% for DPO, and 94.9% for RLVR.

This result changes the interpretation of any high probe score. Hidden states are functions of the prompt and can encode prompt identity, while the activation vector has far more dimensions than there are prompt groups. Consequently, a probe may recover nearly all reward variance by identifying which prompt produced the activation, without representing rollout-specific future success. Repeated generations increase the number of rows from six to 30 per checkpoint, but when their outcomes are constant within prompt they do not create the counterfactual variation required to distinguish prompt recognition from value estimation.

4.3 Constraint Outcomes Were Sparse and Highly Specific

Appendix Figure 3 shows the observed constraint pass rates. The exclude-word and palindrome constraints were passed in every rollout for every checkpoint. In contrast, JSON formatting, first-word, same-start/end, required-last-word, paragraph-first-word, exact-word-count, and counting-composition constraints were never passed in this sample.

The clearest post-training increase occurred for the requirement that a specified word appear exactly once: its pass rate was 0% for Base, 50% for SFT, 50% for DPO, and 60% for RLVR. The lowercase requirement showed the opposite pattern, with 80% for Base and 0% for all three post-trained checkpoints. Because constraint identity is nearly confounded with prompt identity in this sample, these values describe the six selected prompts rather than general constraint-level capabilities.

4.4 Observable Prefix Status Was a Strong Competing Explanation

The external verifier was applied to each saved partial response as a diagnostic of information already visible in the text. At 25% of generation, the prefix score exactly equaled the eventual terminal score for 93.3% of Base responses, 83.3% of SFT responses, 83.3% of DPO responses, and 80.0% of RLVR responses. The corresponding pooled correlations with terminal reward were 0.964, 0.592, 0.500, and 0.527. By 50%, exact agreement ranged from 90.0% to 96.7%.

These large values should not be interpreted as successful early prediction. They are inflated by prompt-specific constraints and by the many cases in which both prefix and terminal reward are zero. Instead, they demonstrate the main identification problem: a predictor can achieve impressive agreement without recovering a latent belief about future success. A meaningful activation result would have to improve on this observable-prefix information on prompts with genuine within-prompt outcome variation. The current held-out split cannot perform that comparison.

4.5 Held-Out Reward Decodability Was Not Evaluable

The fixed split assigned prompts 335-54 and 935-9 to the test set. Both prompts received terminal reward zero in all five rollouts from all four checkpoints. Consequently, test-target variance was zero for every probe evaluation. Held-out R^2 and rank correlation were therefore undefined across all 256 checkpoint, layer, fraction, and feature-set combinations.

Mean squared error can still be calculated against a constant target, but it does not measure discrimination between successful and unsuccessful held-out outcomes. The shuffled-label null likewise cannot repair the missing outcome variation. Accordingly, this experiment yields no evidence for or against linear terminal-reward decodability. It instead identifies a data-coverage failure in the pilot evaluation.

5 Discussion

The experiment does not currently support its central hypothesis that trajectory-specific internal state provides a useful estimate of future terminal reward. This is not because a probe obtained a low score. It is because the data do not separate the proposed signal from simpler explanations. Prompt identity almost determines reward, partial text exposes much of the same structure, the held-out target is constant, and only a handful of prompt-by-checkpoint cells contain different outcomes across rollouts.

This distinction is essential for interpreting probes. A 100% training score would be unsurprising in a setting with 20 training responses, 4,096–16,384 activation features, and labels that are nearly deterministic functions of six prompt identities. Such a score would show that the representation contains enough information to reconstruct the dataset labels. It would not show that the model maintains a value estimate, that the decoded feature precedes the outcome, or

that the model uses it to select behavior. Even a high held-out score can remain ambiguous if the split preserves the same constraint templates or if an observable prefix baseline performs equally well.

The Base checkpoint is an experimental reference, not a sufficient statistical control for this inference. A stronger Base-to-RLVR probe does not by itself show that RL created or amplified a reward representation: RL may alter prompt encoding, residual scale, generated text, or separability without improving a model-internal estimate of future reward. Direct comparison of RMS distances across checkpoints is especially limited because the checkpoints inhabit different representation spaces and generate different token sequences.

The confidence results are therefore narrower than a claim that confidence is anti-faithful. Post-trained models were more confident on average, and their pooled confidence–reward association became negative late in generation. But prompt centering changed several coefficients substantially or reversed their sign. The supported conclusion is that confidence was not a stable surrogate for IFEvalG reward in this sample. The study does not establish a general negative relationship between confidence and instruction following.

Likewise, the aggregate reward ordering is not evidence that post-training improved instruction following generally. Four prompts were unchanged between Base and RLVR, one improved, and one worsened. All responses were length censored, and the prompts came from an RLVR training-source distribution. The comparison therefore cannot distinguish memorization, elicitation, or generalization.

6 Future Work

6.1 Estimate Value from Branched Continuations

The next experiment will make a prefix, rather than a completed response, the unit of analysis. At fixed token or semantic-step boundaries, I will save a prefix and its hidden state, then sample multiple independent continuations from that identical prefix. Their mean terminal outcome estimates

$$\widehat{V}(x, y_{\leq t}) = \frac{1}{K} \sum_{k=1}^K R_T^{(k)} \approx \mathbb{E}[R_T \mid x, y_{\leq t}]. \quad (11)$$

This separates the expected prospects of the prefix from randomness in one sampled future. Prefix lengths will be fixed in advance rather than defined as a fraction of the eventual response. The primary test is incremental: does a model-internal feature predict continuation value on held-out prompt and constraint families after accounting for the instruction, current partial verifier status, length, token identity, and policy confidence? Current state, simple pooled trajectory features, and compact residual-geometry summaries will be evaluated separately before combined models are considered.

6.2 Language-Conditioned Sequential Benchmarks

IFEval offers exact rewards but a narrow set of mostly lexical constraints. I plan to add reasoning and agentic tasks in which intermediate actions have clear consequences. MemoryArena is a useful model: its interdependent multi-session tasks require an agent to acquire information and feedback, store or revise memory, and use that history in later decisions across web navigation, preference-constrained planning, progressive search, and formal reasoning [11].

For autoregressive generation, the complete prompt and token prefix can be treated as an MDP state because it is sufficient to define the model’s next token distribution. At a semantic-step level I will define

$$s_t = (g, o_{\leq t}, m_t), \quad a_t = \text{next language or tool action}, \quad (12)$$

where g is the natural-language goal, $o_{\leq t}$ is observed interaction history, and m_t is the agent’s maintained memory. In environments such as MemoryArena, the true world state is only partially observed, so this is more properly a POMDP whose history or memory supplies the agent’s belief state. Evaluation will record task success, verifier-defined step progress, memory updates, and continuation value at the same action boundaries. This tests whether the internal representation tracks not only textual compliance but the future usefulness of information and actions.

6.3 From Decoding to Reward Sensitivity and Control

The representation-level target will be explicitly conditioned on language,

$$V_\phi(h_{\ell,t}, g) \approx \mathbb{E}[R_T \mid h_{\ell,t}, g]. \quad (13)$$

If this function transfers across held-out goals, its gradient $\nabla_{h_{\ell,t}} V_\phi$ gives a local direction in residual space along which predicted return changes most. Backpropagating that estimate across layers can identify which components and earlier positions the fitted value is sensitive to. This is the sense in which one might “take the derivative back” through an LLM representation. The gradient is initially only an attribution of the fitted estimator; it becomes evidence about the model’s mechanism only if controlled movement along the direction changes continuation rewards while matched orthogonal and shuffled directions do not.

I will therefore separate three claims. Held-out incremental prediction tests whether value information is recoverable. Comparisons across intermediate pretraining revisions and the released SFT, DPO, and RLVR stages, evaluated on identical teacher-forced prefixes, test when an outcome-relevant signal emerges and whether post-training exposes or reorganizes one already present in Base. Activation intervention followed by fresh continuation sampling tests whether the direction participates causally in behavior. This sequence connects internal measurement to reward design without equating probe weights with the true reward gradient.

Acknowledgments

GPT-5.6 Sol was used for implementation support, debugging, figure generation, and manuscript editing.

A Complete Measurement Audit for One Prompt

This appendix follows one real prompt through generation, verification, policy statistics, and activation extraction. It uses rollout index 0 for prompt IF_multi_constraints_upto5_filtered-request-639-88 in each model. The examples document the measurement procedure; four responses to one prompt do not estimate a model-stage effect.

A.1 Prompt and Executable Conditions

The saved prompt text was:

“Can you list, explain, and give every individual reaction involved in the process of human digestion from ingestion to excretion? There should be 4 paragraphs. Paragraphs and only paragraphs are separated with each other by two new lines as if it was ‘\n\n’ in python. Paragraph 3 must start with word *art*. Include keyword *being* in your response. The last word of your response should be the word *stroke*.”

The dataset’s `ground_truth` field was compiled into three IFEvalG calls: `length_constraints:nth_paragraph_first_word` with four paragraphs, paragraph 3, and first word *art*; `keywords:word_once` with keyword *being*; and `last_word:last_word_answer` with final word *stroke*. These executable conditions, rather than a semantic assessment of the requested digestion explanation, determine R_T .

A.2 Four Actual Generations

All four saved generations ended at the 128-generated-token cap.

Base, rollout 0.

```
"<h2> <span style="color: #3b5998; font-weight: bold;">The Journey of Human Digestion</span> </h2> <p>
<span style="color: #3b5998; font-weight: bold;">Ingestion</span> </p> <p> <span style="color: #3b5998; font-
weight: bold;">Ingestion</span> </p> <p> <span style="color: #3b5998; font-weight: bold;">Art of Swallowing</span>
</p> <p> <span"
```

SFT, rollout 0.

"Okay, the user wants me to list, explain, and give every individual reaction involved in human digestion from ingestion to excretion. They specified four paragraphs, each separated by two newlines. The third paragraph must start with "art," and I need to include the keyword "being" somewhere. The last word should be "stroke." Let me break this down.

First, I need to recall the steps of human digestion. It starts with ingestion, then moves through mechanical and chemical processes. The first paragraph should probably cover the mouth, like saliva, mastication. Mention enzymes like amylase and the role of being in the mouth"

DPO, rollout 0.

"Okay, the user wants me to list, explain, and detail every individual reaction involved in human digestion from ingestion to excretion, split into four paragraphs with two newlines between them. The third one has to start with "Art" and include the keyword "being". Plus, the last word must be "stroke". Hmm, let me break this down.

First, I need to recall the digestion process step by step. Starting with ingestion in the mouth—salivation, chewing, maybe enzymes like amylase. Then moving to the stomach with gastric juices, acid and pepsin. Next, the small intestine where bile, pancreatic"

RLVR, rollout 0.

"Okay, the user wants me to list, explain, and give every individual reaction involved in human digestion from ingestion to excretion. They specified four paragraphs, each separated by two newlines. The third paragraph must start with "Art" and include the keyword "being". The last word has to be "stroke". Hmm, let me break this down.

First, I need to recall the stages of digestion. Starting with ingestion in the mouth, then moving through mechanical and chemical processes. The mouth has saliva, enzymes like amylase. Then food enters the stomach, where acids and pepsin break down proteins. Next, the small"

A.3 Verifier Results and Terminal Reward

Table 2 contains the saved Boolean result of every verifier call. DPO and RLVR passed only the exactly-once keyword check. SFT used the word *being* twice and therefore failed that check. Base used HTML paragraph tags, but the verifier’s required blank-line paragraph format was not satisfied. None of the capped generations ended with *stroke*.

Table 2. Actual constraint results for rollout 0 of prompt 639-88.

Executable condition	Base	SFT	DPO	RLVR
Four paragraphs; paragraph 3 starts with <i>art</i>	No	No	No	No
Keyword <i>being</i> appears exactly once	No	No	Yes	Yes
Final word is <i>stroke</i>	No	No	No	No
R_T	0	0	1/3	1/3

For completeness, all five terminal rewards for this prompt were Base $[0, 0, 0, 0, 0]$, SFT $[0, 1/3, 1/3, 1/3, 1/3]$, DPO $[1/3, 1/3, 1/3, 1/3, 1/3]$, and RLVR $[1/3, 1/3, 1/3, 1/3, 1/3]$. This is the source of the model-stage means 0, 0.267, 0.333, and 0.333 for this prompt.

A.4 Extracting Residual States

The completed run replayed each prompt plus its completed generation in `activations.py`. Generated-token offsets 31, 63, 95, and 127 in the saved zero-based metadata correspond to generated tokens 32, 64, 96, and 128, or 25%, 50%, 75%, and 100% of these capped responses. At every selected position it saved the contextual residual-stream vector after transformer blocks 8, 16, 24, and 32. Each vector has 4,096 coordinates. It is the state at that token position after attending to the prompt and preceding text; it is not an embedding of the token in isolation. Across the whole run this produced $120 \times 4 \times 4 = 1,920$ vectors.

For the RLVR response above, Table 3 identifies the visible text at each extraction boundary. The partial-verifier score was computed later by applying the same three checks to the text visible at that boundary; it is an observable-text baseline, not a model value estimate.

Table 3. Actual layer-32 trajectory summaries for RLVR rollout 0. C_t is geometric-mean generated-token probability and D_t is residual movement from the 25% state.

Fraction	Token	Visible text ends with	Partial verifier	C_t / D_t
25%	32	<i>each</i>	0	0.928 / 0.000
50%	64	<i>Hmm</i>	1/3	0.923 / 1.903
75%	96	<i>processes</i>	1/3	0.865 / 1.810
100%	128	<i>small</i>	1/3	0.824 / 1.894

The vectors themselves are too large to print. Table 4 gives an auditable scalar summary, the root-mean-square coordinate value $\|h\|_2 / \sqrt{4096}$, computed directly from each saved vector for the four example rollouts. RMS magnitude is not value and has no intrinsically desirable direction.

Table 4. Residual-vector RMS for the four rollout-0 examples.

Model	Fraction	Layer 8	Layer 16	Layer 24	Layer 32
Base	25%	0.059	0.073	0.132	1.430
Base	50%	0.138	0.257	0.584	1.567
Base	75%	0.150	0.281	0.543	1.571
Base	100%	0.162	0.300	0.655	1.436
SFT	25%	0.118	0.239	0.564	1.406
SFT	50%	0.103	0.239	0.511	1.386
SFT	75%	0.149	0.270	0.622	1.377
SFT	100%	0.149	0.264	0.570	1.398
DPO	25%	0.102	0.249	0.620	1.390
DPO	50%	0.089	0.273	0.647	1.387
DPO	75%	0.189	0.313	0.604	1.382
DPO	100%	0.204	0.306	0.594	1.380
RLVR	25%	0.118	0.241	0.571	1.387
RLVR	50%	0.146	0.299	0.644	1.376
RLVR	75%	0.132	0.278	0.627	1.379
RLVR	100%	0.135	0.240	0.543	1.401

A.5 Policy Confidence and Entropy

For each generated token, the model’s final hidden state at the preceding position was passed through the LM head. The extraction code computed the sampled token’s log probability, the entropy of the full next-token distribution, and its top-1 probability. The saved JSON retains eight summaries at each of the four fractions: prefix mean token log probability, its exponentiated geometric-mean probability, prefix sequence log probability, prefix mean entropy, prefix mean top-1 probability, and the corresponding three last-token statistics. Tables 5 and 6 report every saved policy statistic for the example rollouts.

Table 5. Actual prefix-aggregated policy statistics. $\overline{\log p}$ is mean sampled-token log probability, C is its exponentiated value, $\log p(y_{\leq t})$ is cumulative sequence log probability, $\overline{H}$ is mean entropy, and $\overline{p}_{\max}$ is mean top-1 probability.

Model	Fraction	$\overline{\log p}$	C	$\log p(y_{\leq t})$	$\overline{H}$	$\overline{p}_{\max}$
Base	25%	-0.900	0.406	-28.810	1.743	0.581
Base	50%	-0.566	0.568	-36.222	1.172	0.713
Base	75%	-0.418	0.658	-40.138	0.925	0.779
Base	100%	-0.348	0.706	-44.535	0.777	0.817
SFT	25%	-0.110	0.896	-3.517	0.276	0.908
SFT	50%	-0.156	0.855	-10.011	0.311	0.892
SFT	75%	-0.218	0.804	-20.919	0.423	0.853
SFT	100%	-0.399	0.671	-51.022	0.723	0.789
DPO	25%	-0.121	0.886	-3.888	0.175	0.925
DPO	50%	-0.109	0.897	-6.952	0.193	0.925
DPO	75%	-0.165	0.848	-15.815	0.260	0.904
DPO	100%	-0.215	0.806	-27.562	0.373	0.879
RLVR	25%	-0.075	0.928	-2.397	0.164	0.941
RLVR	50%	-0.080	0.923	-5.132	0.184	0.936
RLVR	75%	-0.145	0.865	-13.893	0.303	0.897
RLVR	100%	-0.193	0.824	-24.707	0.421	0.866

Table 6. Actual policy statistics for the last generated token at each sampled fraction.

Model	Fraction	Last $\log p$	Last entropy	Last $p_{\max}$
Base	25%	-0.009	0.066	0.991
Base	50%	-0.027	0.266	0.974
Base	75%	-0.140	0.857	0.869
Base	100%	-0.005	0.046	0.995
SFT	25%	-0.082	0.337	0.921
SFT	50%	-0.000	0.002	1.000
SFT	75%	-0.002	0.020	0.998
SFT	100%	-3.204	3.732	0.362
DPO	25%	-0.070	0.317	0.932
DPO	50%	-0.000	0.000	1.000
DPO	75%	-0.430	1.607	0.650
DPO	100%	-0.662	1.890	0.516
RLVR	25%	-0.015	0.083	0.985
RLVR	50%	-0.045	0.212	0.956
RLVR	75%	-0.332	0.859	0.717
RLVR	100%	-0.040	0.278	0.961

A.6 Constraint-Level Results

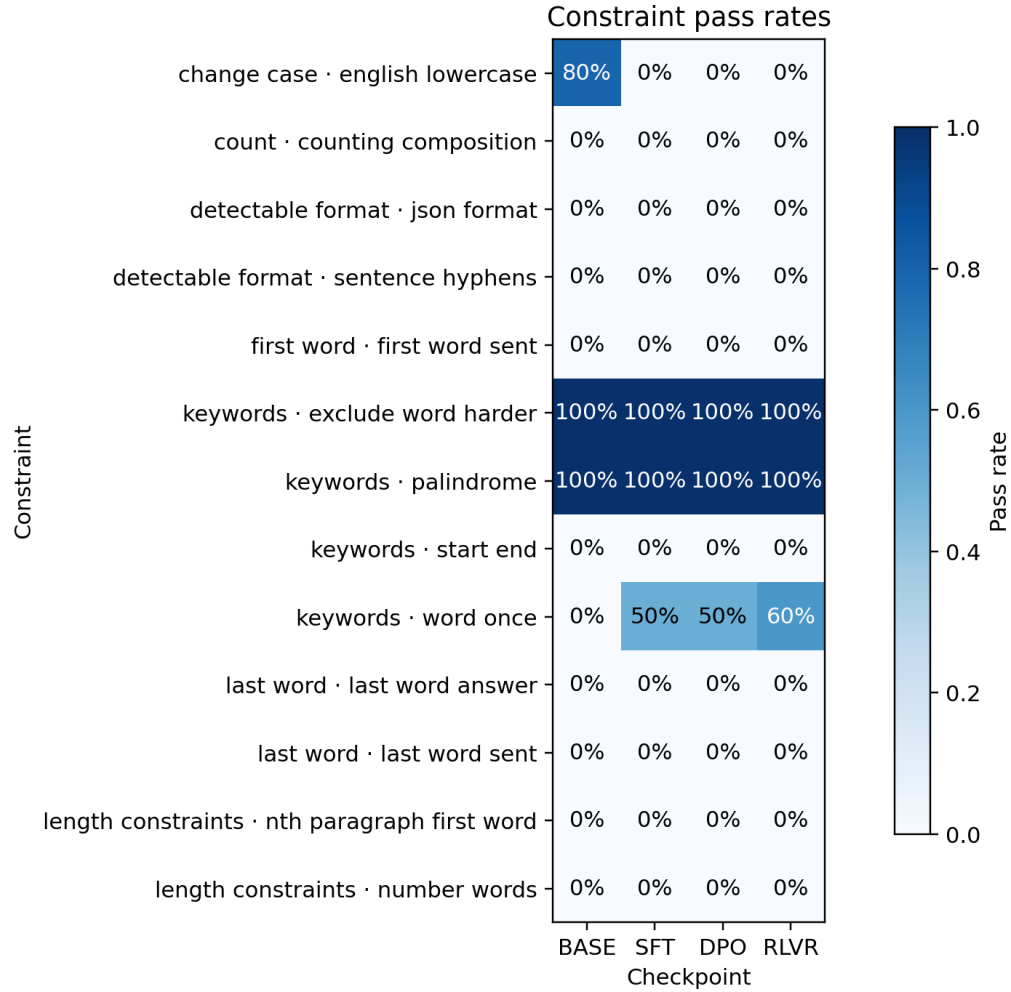


Fig. 3. Observed pass rate for each verifier constraint and model variant. Each cell is the fraction of evaluated instances that passed. Counts are omitted because most constraint IDs occur in only one of the sampled prompts.

References

- [1] Shivam Agarwal, Zimin Zhang, Lifan Yuan, Jiawei Han, and Hao Peng. 2025. The Unreasonable Effectiveness of Entropy Minimization in LLM Reasoning. *arXiv preprint arXiv:2505.15134* (2025).
- [2] Allen Institute for AI. 2025. Dolci-Think-RL-7B Dataset Card. <https://huggingface.co/datasets/allenai/Dolci-Think-RL-7B>. Accessed 2026-09-05.
- [3] Allen Institute for AI. 2025. OLMo 3 7B Think SFT Model Card. <https://huggingface.co/allenai/Olmo-3-7B-Think-SFT>. Accessed 2026-09-05.
- [4] Allen Institute for AI. 2025. Open Instruct. <https://github.com/allenai/open-instruct>. Accessed 2026-09-05.
- [5] Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. 2025. SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-training. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 10818–10838.

- [6] Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. 2025. The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models. *arXiv preprint arXiv:2505.22617* (2025).
- [7] Zhichen Dong, Zhanhui Zhou, Zhixuan Liu, Chao Yang, and Chaochao Lu. 2025. Emergent Response Planning in LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*.
- [8] Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. 2021. Amnesic Probing: Behavioral Explanation with Amnesic Counterfactuals. *Transactions of the Association for Computational Linguistics* 9 (2021), 160–175. doi:10.1162/tacl_a_00359
- [9] Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. 2025. Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs. arXiv:2503.01307 [cs.CL] doi:10.48550/arXiv.2503.01307
- [10] Dirk Groeneveld et al. 2024. OLMo: Accelerating the Science of Language Models. *arXiv preprint arXiv:2402.00838* (2024).
- [11] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. 2026. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks. *arXiv preprint arXiv:2602.16313* (2026).
- [12] John Hewitt, Kavin Ethayarajh, Percy Liang, and Christopher Manning. 2021. Conditional Probing: Measuring Usable Information Beyond a Baseline. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 1626–1639. doi:10.18653/v1/2021.emnlp-main.122
- [13] John Hewitt and Percy Liang. 2019. Designing and Interpreting Probes with Control Tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, 2733–2743. doi:10.18653/v1/D19-1275
- [14] Ling kai Kong, Haorui Wang, Wenhao Mu, Yuanqi Du, Yuchen Zhuang, Yifei Zhou, Yue Song, Rongzhi Zhang, Kai Wang, and Chao Zhang. 2024. Aligning Large Language Models with Representation Editing: A Control Perspective. In *Advances in Neural Information Processing Systems*.
- [15] Gerson Kroiz, Aditya Singh, Senthoooran Rajamanoharan, and Neel Nanda. 2026. Principled Interpretability of Reward Hacking in Closed Frontier Models. AI Alignment Forum. <https://www.alignmentforum.org/posts/A67SbpTjuXEhK8Cvo/principled-interpretability-of-reward-hacking-in-closed> Accessed 2026-09-12.
- [16] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s Verify Step by Step. *arXiv preprint arXiv:2305.20050* (2023).
- [17] Andrew Y. Ng and Stuart J. Russell. 2000. Algorithms for Inverse Reinforcement Learning. In *Proceedings of the Seventeenth International Conference on Machine Learning*. 663–670.
- [18] Hoang Cuong Nguyen, Mark Dras, and Usman Naseem. 2026. Beyond Shallow Alignment: How Post-Training Methods Determine Refusal Circuits and Steering Robustness. *arXiv preprint arXiv:2609.03887* (2026). Accepted at EMNLP 2026.
- [19] Mihir Prabhudesai, Lili Chen, Alex Ippoliti, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. 2025. Maximizing Confidence Alone Improves Reasoning. *arXiv preprint arXiv:2505.22660* (2025).
- [20] Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shao lei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. 2026. Spurious Rewards: Rethinking Training Signals in RLVR. arXiv:2506.10947 [cs.AI] doi:10.48550/arXiv.2506.10947
- [21] Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. 2024. Text2Reward: Reward Shaping with Language Models for Reinforcement Learning. In *International Conference on Learning Representations*.
- [22] Dan Zhang, Sining Zhoubian, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. 2024. ReST-MCTS*: LLM Self-Training via Process Reward Guided Tree Search. *Advances in Neural Information Processing Systems* 37 (2024).
- [23] Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, Guoli Jia, Pengfei Li, Yu Fu, Xingtai Lv, Yuchen Zhang, Sihang Zeng, Shang Qu, Haozhan Li, Shijie Wang, Yuru Wang, Xinwei Long, Fangfu Liu, Xiang Xu, Jiaze Ma, Xuekai Zhu, Ermo Hua, Yihao Liu, Zonglin Li, Huayu Chen, Xiaoye Qu, Yafu Li, Weize Chen, Zhenzhao Yuan, Junqi Gao, Dong Li, Zhiyuan Ma, Ganqu Cui, Zhiyuan Liu, Biqing Qi, Ning Ding, and Bowen Zhou. 2025. A Survey of Reinforcement Learning for Large Reasoning Models. arXiv:2509.08827 [cs.CL] doi:10.48550/arXiv.2509.08827
- [24] Yan zhi Zhang, Zhaoxi Zhang, Haoxiang Guan, Yilin Cheng, Yitong Duan, Chen Wang, Yue Wang, Shuxin Zheng, and Jiyan He. 2025. No Free Lunch: Rethinking Internal Feedback for LLM Reasoning. *arXiv preprint arXiv:2506.17219* (2025).
- [25] Xuandong Zhao, Zhewei Kang, Aosong Feng, Sergey Levine, and Dawn Song. 2025. Learning to Reason without External Rewards. *arXiv preprint arXiv:2505.19590* (2025).
- [26] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-Following Evaluation for Large Language Models. *arXiv preprint arXiv:2311.07911* (2023).
- [27] Brian D. Ziebart, Andrew L. Maas, J. Andrew Bagnell, and Anind K. Dey. 2008. Maximum Entropy Inverse Reinforcement Learning. In *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*. 1433–1438.